Assignment 5

NAME : YUVARAJ R
REG NO: 26MCS1019

Q1.

Problem Statement

Emma is working on a navigation system that needs to find the distance between two nodes in a Binary Search Tree (BST). Implement a function to help Emma calculate the distance between two nodes.

Example

Input:
7
20 10 30 5 15 25 35
5 35
Output:
4
Explanation:
The BST for the nodes is

The nodes for which we need to find the distance are 5 and 35.
The path from 5 to 35 is, from 5 to 10 (1 step), from 10 to 20 (1 step), from 20 to 30 (1 step), from 30 to 35 (1 step).
Thus, the total distance is 4 steps.

Input Format

The first line contains an integer **n**, representing the number of nodes to be inserted into the BST.
The second line consists of **n** space-separated integers, representing the values to be inserted.
The third line consists of two integers **n1** and **n2**, representing the values of the two nodes.

Output Format

The output prints the distance between the two given nodes.

Refer to the sample output for formatting specifications.

Constraints

1 ≤ n ≤ 25
1 ≤ tree elements, n1, n2 ≤ 100

Sample Input Sample Output

```
7
20 10 30 5 15 25 35
```

```
5 35
4
```

Sample Input Sample Output

```
5
56 64 8 9 43
8 43
2
```

CODE :

```
#include <stdio.h>

#include <stdlib.h>


struct Node {

   int data;

   struct Node *left;

   struct Node *right;

};


// Create a new node

struct Node* createNode(int value) {

   struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));


   newNode->data = value;

   newNode->left = NULL;

   newNode->right = NULL;


   return newNode;

}
```

```
// Insert a value into the BST
struct Node* insert(struct Node* root, int value) {
    if (root == NULL) {
        return createNode(value);
    }

    if (value < root->data) {
        root->left = insert(root->left, value);
    }
    else {
        root->right = insert(root->right, value);
    }

    return root;
}

// Find the Lowest Common Ancestor
struct Node* findLCA(struct Node* root, int n1, int n2) {

    while (root != NULL) {

        // Both nodes are smaller → go left
        if (n1 < root->data && n2 < root->data) {
            root = root->left;
        }
```

```
        // Both nodes are greater → go right
        else if (n1 > root->data && n2 > root->data) {
            root = root->right;
        }

        // Otherwise, this is the LCA
        else {
            return root;
        }
    }

    return NULL;
}

// Find distance from root to a given value
int findDistance(struct Node* root, int value) {
    int distance = 0;

    while (root != NULL) {

        if (value == root->data) {
            return distance;
        }
```

```
        if (value < root->data) {
            root = root->left;
        }
        else {
            root = root->right;
        }

        distance++;
    }

    return -1;
}

int main() {
    int n;
    scanf("%d", &n);

    struct Node* root = NULL;

    // Build the BST
    for (int i = 0; i < n; i++) {
        int value;
        scanf("%d", &value);

        root = insert(root, value);
```

```
    }

    int n1, n2;
    scanf("%d %d", &n1, &n2);

    // Find LCA
    struct Node* lca = findLCA(root, n1, n2);

    // Find distances from LCA
    int d1 = findDistance(lca, n1);
    int d2 = findDistance(lca, n2);

    printf("%d\n", d1 + d2);

    return 0;
}
```

Q2.

Problem Statement

Jay is studying binary search trees (BSTs) as part of his computer science curriculum. He is particularly interested in understanding operations like insertion and deletion in a BST.

Jay wants to write a program that takes a sequence of characters as input, constructs a BST using those characters, deletes the minimum element from the BST, and then outputs the elements of the tree in sorted order.

Help Jay by designing a program that achieves this.

Input Format

The first line of input contains an integer n, denoting the number of characters to be inserted into the BST.

The second line contains n space-separated characters, each character representing a node to be inserted into the BST.

Output Format

The output displays a single line displaying the space-separated characters of the BST in sorted order after deleting the minimum element in the format:

"Inorder traversal after deleting the minimum element:"

"[char1] [char2] ...[char_n]"

Refer to the sample output for the formatting specifications.

Constraints

The given test cases will fall under the following constraints:

1 ≤ n ≤ 10

Characters are uppercase English alphabets.

All characters in the input are distinct.

Sample Input Sample Output

```
6
K J M V H F
Inorder traversal after deleting the minimum element:
H J K M V
```

Sample Input Sample Output

```
4
T B C A
Inorder traversal after deleting the minimum element:
B C T
```

CODE :

```
#include <stdio.h>

#include <stdlib.h>



struct Node {

   char data;

   struct Node *left;

   struct Node *right;

};



/* Create a new node */
```

```
struct Node* createNode(char value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));

    newNode->data = value;
    newNode->left = NULL;
    newNode->right = NULL;

    return newNode;
}

/* Insert a character into the BST */
struct Node* insert(struct Node* root, char value) {
    if (root == NULL) {
        return createNode(value);
    }

    if (value < root->data) {
        root->left = insert(root->left, value);
    } else {
        root->right = insert(root->right, value);
    }

    return root;
}
```

```
/* Delete the minimum element */
struct Node* deleteMin(struct Node* root) {
    if (root == NULL) {
        return NULL;
    }

    /* No left child means this is the minimum */
    if (root->left == NULL) {
        struct Node* temp = root->right;
        free(root);
        return temp;
    }

    root->left = deleteMin(root->left);

    return root;
}

/* Inorder traversal */
void inorder(struct Node* root) {
    if (root == NULL) {
        return;
    }

    inorder(root->left);
```

```
    printf("%c ", root->data);

    inorder(root->right);
}

int main() {
    int n;
    scanf("%d", &n);

    struct Node* root = NULL;

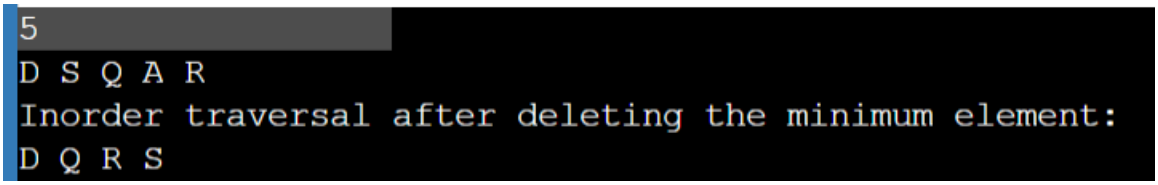
    for (int i = 0; i < n; i++) {
        char ch;
        scanf(" %c", &ch);

        root = insert(root, ch);
    }

    root = deleteMin(root);

    printf("Inorder traversal after deleting the minimum element:\n");
    inorder(root);
    return 0;
}
```

OUTPUT :



Q3.

Problem Statement

Jasmina needs a program that performs a postorder traversal on a binary tree constructed from an array of integers. The program should take an integer n as input, followed by n integers representing the elements of the array.

It constructs a binary tree using the array elements in level order and then performs a postorder traversal on the constructed binary tree. Finally, it prints the sequence of nodes visited during the postorder traversal.

Input Format

The first line consists of an integer **n**, representing the number of elements in the array. The second line consists of **n** space-separated integers representing the elements of the array.

Output Format

The output prints a single line containing the postorder traversal of the constructed binary tree.

Refer to the sample output for the formatting specifications.

Constraints

The input integers should be unique.

1 ≤ n ≤ 20

1 ≤ array elements ≤ 1500

Sample Input Sample Output

```
6
9 8 5 4 3 1
4 3 8 1 5 9
```

Sample Input Sample Output

```
5
1 3 5 7 9
7 9 3 5 1
```

CODE :

```
#include <stdio.h>

void postorder(int arr[], int i, int n) {
    if (i >= n)
```

```
        return;

    // Left subtree
    postorder(arr, 2 * i + 1, n);

    // Right subtree
    postorder(arr, 2 * i + 2, n);

    // Root
    printf("%d ", arr[i]);
}

int main() {
    int n;
    scanf("%d", &n);

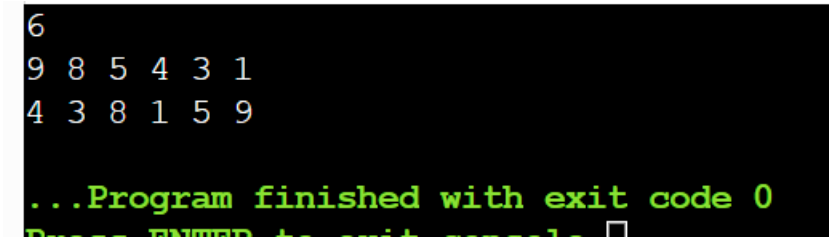
    int arr[20];

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    postorder(arr, 0, n);

    return 0;
}
```

OUTPUT :



Q4.

Problem Statement

Liam needs to build a Binary Search Tree (BST) from user input and determine the in-order successor of a specific node. Write a program to help Liam construct the BST, then find and display the in-order successor of the target node.

Assist Liam with a suitable program.

In the above diagram, the in-order successor of 8 is 10, the in-order successor of 10 is 12 and the in-order successor of 14 is 20.

Input Format

The first line of input consists of an integer **n** representing the number of nodes in the binary search tree.

The second line of input consists of **n** integers representing the values of the nodes in the binary search tree.

The third line of input consists of an integer **T**, representing the value for which to find the in-order successor.

Output Format

The output prints "In-order Successor of T is X" where X is an integer representing the in-order successor of the node with the target (T) value.

If an in-order successor doesn't exist, print "In-order Successor doesn't exist".

Refer to the sample output for the formatting specifications.

Constraints

In this scenario, the given test cases will fall under the following constraints:

3 ≤ n ≤ 20

1 ≤ node values ≤ 1000

1 ≤ T ≤ 1000

Sample Input Sample Output

```
7
20 8 22 4 12 10 14
8
In-order Successor of 8 is 10
```

Sample Input Sample Output

```
7
20 8 22 4 12 10 14
14
In-order Successor of 14 is 20
```

Sample Input Sample Output

```
5
10 5 15 3 7
15
In-order Successor doesn't exist
```

Code :

```
#include <stdio.h>



int main()

{
```

```
int n, T;
int arr[20];

scanf("%d", &n);

for (int i = 0; i < n; i++)
{
   scanf("%d", &arr[i]);
}

scanf("%d", &T);

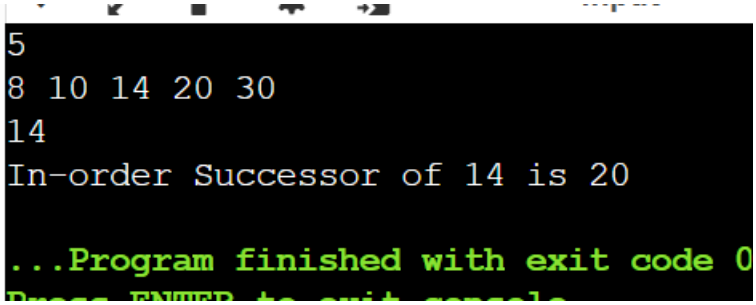
// Find the smallest value greater than T
int successor = -1;

for (int i = 0; i < n; i++)
{
   if (arr[i] > T)
   {
      if (successor == -1 || arr[i] < successor)
      {
         successor = arr[i];
      }
   }
}
```

```
    if (successor == -1)
    {
        printf("In-order Successor doesn't exist");
    }
    else
    {
        printf("In-order Successor of %d is %d", T, successor);
    }

    return 0;
}
```

OUTPUT :



Q5.

Problem Statement

Jamie needs to build a binary tree to represent a family hierarchy. Each node represents a family member, and nodes are added sequentially from top to bottom, left to right, based on user input. Jamie must then implement a function to display the family tree using level-order traversal.

Input Format

The first line consists of an integer **n**, representing the number of nodes in the binary tree.

The second line consists of **n** space-separated integers, representing the elements of the binary tree.

Output Format

The output displays a single line containing the level order traversal of the constructed binary tree.

Refer to the sample output for the formatting specifications.

Constraints

1 ≤ n ≤ 20

1 ≤ elements ≤ 100

The binary tree is not empty.

Sample Input Sample Output

```
5
18 2 37 4 5
18 2 37 4 5
```

Sample Input Sample Output

```
8
15 10 20 8 12 16 25 30
15 10 20 8 12 16 25 30
```

CODE :

```
#include <stdio.h>

int main()
{
    int n;
    int arr[20];

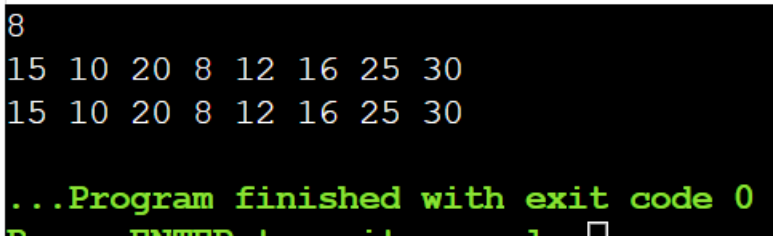
    scanf("%d", &n);

    for (int i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }

    for (int i = 0; i < n; i++)
    {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

OUTPUT :


8
15 10 20 8 12 16 25 30
15 10 20 8 12 16 25 30
...Program finished with exit code 0